Applied Type Theory

Лекция 4: ADT и алгебра типов

Воронов Михаил Сергеевич

ВМК МГУ · Осень 2026

Тип результата вычисления

Как описать результат операции, которая возвращает **значение** типа A или **ошибку** типа E?

```
type ('a, 'e) response =
  | Success of 'a
  | Failure of 'e
```

Result $A\,E \cong A + E$

Конструктор `Success` хранит результат типа A, а `Failure` — описание ошибки типа E.

Тип Option$A \times$ Option$E$ допускает также пары (None, None) и (Some a, Some e).

План лекции

1 **Произведения, суммы и правила**

2 Мощности и изоморфизмы типов

3 Рекурсивные данные и свёртки

Как задаётся новый тип

Формирование

Из каких типов можно построить новый тип

Введение

Какие конструкторы создают его значения

Устранение

Как использовать значение: применить, разобрать, спроецировать

Вычисление

Что происходит, когда устранение встречает конструктор

Согласованность правил введения, устранения и вычисления устанавливается при доказательстве свойств исчисления.

Расширение грамматики простых типов

$$\sigma, \tau ::= \alpha \mid \sigma \to \tau \mid \sigma \times \tau \mid \sigma + \tau \mid \mathbf{1} \mid \mathbf{0}$$

Произведение хранит **обе компоненты**, а сумма — **один из вариантов** вместе с его меткой.

$\mathsf{Bool} \cong \mathbf{1} + \mathbf{1} \qquad \mathsf{Option} A \cong \mathbf{1} + A$

Рекурсивные определения списков и деревьев введём отдельно.

Поскольку стрелка связывает слабее суммы и произведения, группировку типов уточняем скобками.

Произведение: построение и проекции

Введение пары

$$\frac{\Gamma \vdash M : \sigma \qquad \Gamma \vdash N : \tau}{\Gamma \vdash (M, N) : \sigma \times \tau}$$

Устранение произведения

$$\frac{\Gamma \vdash P : \sigma \times \tau}{\Gamma \vdash \pi_1 P : \sigma} \qquad \frac{\Gamma \vdash P : \sigma \times \tau}{\Gamma \vdash \pi_2 P : \tau}$$

Проекции получают компоненты **по их позиции**.

Вычисления с произведением

Шаги call-by-value

$$\pi_1(V, W) \to_{\mathrm{v}} V \qquad \pi_2(V, W) \to_{\mathrm{v}} W$$

При call-by-value компоненты пары вычисляются слева направо до значений V и W.

Восстановление пары

$$(\pi_1 p, \pi_2 p) =_{\eta} p$$

Это экстенсиональный закон: равенство по поведению в чистом тотальном исчислении.

В языке с эффектами повторное вычисление выражения p может менять поведение.

Сумма: выбор конструктора

$$\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \mathsf{inl}_{\tau} M : \sigma + \tau} \qquad \frac{\Gamma \vdash N : \tau}{\Gamma \vdash \mathsf{inr}_{\sigma} N : \sigma + \tau}$$

Метки inl и inr различают варианты, даже если их содержимое имеет одинаковый тип.

$$\mathsf{inl}_{\mathsf{Bool}}\,\mathsf{true} \;:\; \mathsf{Bool} + \mathsf{Bool} \qquad \mathsf{inr}_{\mathsf{Bool}}\,\mathsf{true} \;:\; \mathsf{Bool} + \mathsf{Bool}$$

Индекс задаёт тип другой ветви и опускается, когда полный тип суммы уже задан.

Сумма: разбор вариантов

$$\frac{\Gamma \vdash L : \sigma + \tau \qquad \Gamma, x : \sigma \vdash M : \rho \qquad \Gamma, y : \tau \vdash N : \rho}{\Gamma \vdash \mathsf{case}\, L\, \mathsf{of}\, \mathsf{inl}\, x \Rightarrow M \mid \mathsf{inr}\, y \Rightarrow N : \rho}$$

В разборе суммы **обе ветви** имеют **один и тот же тип результата**.

Переменная x доступна только в левой ветви, y — только в правой.

Имена параметров выбираются свежими для Γ.

Разбор конструктора и подстановка

$\mathsf{case}\ \mathsf{inl} V\ \mathsf{of}\ \mathsf{inl}\ x \Rightarrow M \mid \mathsf{inr}\ y \Rightarrow N \to_{\mathrm{v}} M[x := V]$

$\mathsf{case}\ \mathsf{inr} W\ \mathsf{of}\ \mathsf{inl}\ x \Rightarrow M \mid \mathsf{inr}\ y \Rightarrow N \to_{\mathrm{v}} N[y := W]$

Подставляем значение в выбранную ветвь, сохраняя **свободу переменных** подставляемого терма.

Закон восстановления суммы

$\mathsf{case}\ L\ \mathsf{of}\ \mathsf{inl}\ x \Rightarrow \mathsf{inl} x \mid \mathsf{inr}\ y \Rightarrow \mathsf{inr} y =_{\eta} L$

Последнее равенство относится к чистой тотальной интерпретации.

Единичный тип

$$\Gamma \vdash () : \mathbf{1}$$

У типа $\mathbf{1}$ ровно одно значение.

Устранение через разбор единственного конструктора

$$\frac{\Gamma \vdash L : \mathbf{1} \qquad \Gamma \vdash N : \tau}{\Gamma \vdash \mathsf{case}\, L\, \mathsf{of}\, () \Rightarrow N : \tau}$$

$$\mathsf{case}\, ()\, \mathsf{of}\, () \Rightarrow N \to_{\mathrm{v}} N$$

В OCaml тип `unit` часто служит типом результата операции, выполняемой ради эффекта.

Пустой тип

Ноль конструкторов

Множество значений типа $\mathbf{0}$ пусто.

$$\frac{\Gamma \vdash M : \mathbf{0}}{\Gamma \vdash \mathsf{absurd}_{\tau}\, M : \tau}$$

Правило absurd_{τ} задаёт устранение пустого типа в произвольный тип τ.

$$\mathbf{0} \to A \qquad A \to \mathbf{1}$$

Для каждого A в модели множеств есть ровно одна функция каждого из этих видов.

Вложенный паттерн

$(A \times B) + A \to A$

```
let choose = function
  | Left (x, _) -> x
  | Right z -> z
```

Сначала разбираем **метку суммы**, затем извлекаем первую компоненту пары.

$\lambda q.\mathsf{case}\, q\, \mathsf{of}\, \mathsf{inl}\, p \Rightarrow \pi_1 p \mid \mathsf{inr}\, z \Rightarrow z$

Покрытие вариантов

```
let safe_head = function
  | [] -> None
  | x :: _ -> Some x
```

safe_head : List$A$ → Option$A$

Для списка обрабатываем оба конструктора [] и x :: xs, а при расширении типа добавляем ветви для новых вариантов.

В OCaml предупреждение о пропущенном варианте можно настроить как ошибку сборки.

Покрытие гарантирует выбор подходящей ветви, завершение и поведение которой проверяются отдельно.

Метатеория $\lambda_{\to}$ с новыми типами

Канонические формы новых значений

$\sigma \times \tau$: (V, W) $\qquad$ $\sigma + \tau$: $\mathsf{inl} V$ или $\mathsf{inr} W$ $\qquad$ $\mathbf{1}$: $()$

Сохранение типа

$$\Gamma \vdash M : \tau,\ M \to_{\mathrm{v}} N \quad \Longrightarrow \quad \Gamma \vdash N : \tau$$

Прогресс

Замкнутый типизируемый терм является значением или делает шаг, причём для `case` доказательство использует **полный разбор суммы**.

В чистом исчислении, заданном этими правилами, новые случаи доказательства опираются на инверсию и лемму о подстановке.

План лекции

1 Произведения, суммы и правила

2 Мощности и изоморфизмы типов

3 Рекурсивные данные и свёртки

Два поля: вместе или по отдельности

Нужно хранить пару значений типа A или отметку пустого результата.

$\mathrm{Option}A \times \mathrm{Option}A \qquad \mathrm{Option}(A \times A)$

$(\mathbf{1} + A) \times (\mathbf{1} + A) \cong \mathbf{1} + A + A + A \times A$

$\mathrm{Option}(A \times A) \cong \mathbf{1} + A \times A$

Первый тип допускает два смешанных случая, а второй хранит **целую пару или пустой результат**.

Подсчёт значений конечных типов

$|\mathbf{0}| = 0 \qquad |\mathbf{1}| = 1 \qquad |\mathsf{Bool}| = 2$

$|A + B| = |A| + |B|$

$|A \times B| = |A| \cdot |B|$

$|A \to B| = |B|^{|A|}$

Последняя формула считает все тотальные функции между конечными множествами, различаемые по результатам.

Четыре функции из Bool в Bool

Функция	На false	На true	Описание
$\lambda x.x$	false	true	тождественная
not	true	false	инверсия
λx.true	true	true	постоянная true
λx.false	false	false	постоянная false

$$|\text{Bool} \to \text{Bool}| = 2^2 = 4$$

Термы, задающие одинаковые результаты на всех входах, представляют одну и ту же функцию.

Изоморфизм типов

$A \cong B$

Нужно построить преобразования $f : A \to B$ и $g : B \to A$.

$$g \circ f = \mathrm{id}_A \qquad f \circ g = \mathrm{id}_B$$

Оба преобразования возвращают исходные данные после обхода туда и обратно.

Равенства понимаем экстенсионально в чистой тотальной модели.

Коммутативность произведения

$\mathsf{swap}_{A,B} : A \times B \to B \times A$

$\mathsf{swap}_{A,B}(a, b) = (b, a)$

$\mathsf{swap}_{B,A}(\mathsf{swap}_{A,B}(a, b)) = (a, b)$

Проверка на паре доказывает нужное равенство для всех значений произведения.

$A \times B \cong B \times A$

Дистрибутивность типов

$$A \times (B + C) \cong (A \times B) + (A \times C)$$

$$(a, \mathsf{inl}\, b) \longmapsto \mathsf{inl}(a, b)$$

$$(a, \mathsf{inr}\, c) \longmapsto \mathsf{inr}(a, c)$$

Обратное преобразование переносит общую компоненту a наружу.

Проверяем обе композиции отдельно на **левом и правом конструкторах**.

На практике этот закон помогает перечислить случаи при разборе пары «состояние и событие».

Коммутативное полукольцо

$(S, +, \cdot, 0, 1)$

Сложение: коммутативный моноид

$(x + y) + z = x + (y + z) \qquad x + y = y + x$

$x + 0 = x$

Умножение: коммутативный моноид

$(x \cdot y) \cdot z = x \cdot (y \cdot z) \qquad x \cdot y = y \cdot x$

$x \cdot 1 = x$

Эти законы выполняются для всех $x, y, z \in S$, например для натуральных чисел с обычными сложением и умножением.

Дистрибутивность и поглощающий ноль

Связь двух операций

$$x \cdot (y + z) = x \cdot y + x \cdot z$$

$$(x + y) \cdot z = x \cdot z + y \cdot z$$

Ноль поглощает произведение

$$0 \cdot x = x \cdot 0 = 0$$

Дистрибутивность позволяет раскрывать скобки и выносить общий множитель.

$$(x + 1) \cdot (y + 1) = x \cdot y + x + y + 1$$

Полукольцо типов

Сумма и произведение задают операции над классами $[A]$, объединяющими все типы, изоморфные A.

$$[A] + [B] = [A + B] \qquad [A] \cdot [B] = [A \times B]$$

Ассоциативность: меняем группировку

$$(A + B) + C \cong A + (B + C)$$

$$(A \times B) \times C \cong A \times (B \times C)$$

Коммутативность: переставляем компоненты

$$A + B \cong B + A \qquad A \times B \cong B \times A$$

Изоморфизмы типов становятся равенствами их классов, на которых суммы и произведения образуют **коммутативное полукольцо**.

Ноль и единица в типах

Ноль сложения: пустой вариант суммы

$$A + \mathbf{0} \cong A \qquad \mathsf{inl} a \longleftrightarrow a$$

Единица умножения: единственная компонента

$$A \times \mathbf{1} \cong A \qquad (a, ()) \longleftrightarrow a$$

Поглощение: произведение с пустым типом

$$A \times \mathbf{0} \cong \mathbf{0}$$

Множество таких пар пусто: вторая компонента принадлежит $\mathbf{0}$.

Полином типа данных

Тип $P(A, B)$ описывает данные с обязательным полем A и двумя опциональными полями B.

$$P(A, B) = A \times (\mathbf{1} + B) \times (\mathbf{1} + B)$$

$$P(A, B) \cong A + (A \times B) + (A \times B) + (A \times B \times B)$$

$$P(A, B) \cong A + \mathbf{2} \times A \times B + A \times B^2$$

$\mathbf{2} ::= \mathbf{1} + \mathbf{1} \cong$ Bool $\qquad B^2 ::= B \times B$

Коэффициент $\mathbf{2}$ различает два случая с одним полем B, а степень B^2 хранит оба поля.

Мощность как гомоморфизм

Мощность конечного типа $c([A]) = |A|$ задаёт гомоморфизм полукольца, сохраняющий обе операции, ноль и единицу.

$$c([\mathbf{0}]) = 0 \qquad c([\mathbf{1}]) = 1$$

$$c([A] + [B]) = c([A]) + c([B])$$

$$c([A] \cdot [B]) = c([A]) \cdot c([B])$$

Подставляем мощности в полином типа

$$|P(\mathsf{Bool}, \mathsf{Bool})| = 2 \cdot (1 + 2)^2 = 18$$

Преобразования типов задают взаимно обратные программы, а мощности позволяют считать допустимые значения.

Функции и законы экспонент

$$(A + B \to C) \cong (A \to C) \times (B \to C)$$

Функция на сумме задаётся отдельным обработчиком для каждого варианта.

$$(A \times B \to C) \cong (A \to B \to C)$$

Это каррирование: пару аргументов можно передавать по частям.

$$(A \to B \times C) \cong (A \to B) \times (A \to C)$$

Эти изоморфизмы рассматриваем для чистых тотальных функций, а при работе с эффектами учитываем порядок и кратность вычислений.

План лекции

1 Произведения, суммы и правила

2 Мощности и изоморфизмы типов

3 Рекурсивные данные и свёртки

Рекурсивные данные

$$\mathsf{List}A \cong \mathbf{1} + A \times \mathsf{List}A$$

Список либо пуст, либо содержит голову и меньший список.

$$\mathsf{Tree}A \cong \mathbf{1} + A \times \mathsf{Tree}A \times \mathsf{Tree}A$$

Дерево либо пусто, либо содержит метку и два поддерева.

Индуктивная интерпретация выбирает **конечные структуры**, построенные конструкторами.

Функтор одного слоя

$$F_A(X) = \mathbf{1} + A \times X \qquad \mathsf{List}A = \mu X.F_A(X)$$

В $F_A(X)$ параметр X отмечает рекурсивное продолжение, а A задаёт тип данных элемента.

$$F_A(h)(\mathsf{inl}()) = \mathsf{inl}()$$

$$F_A(h)(\mathsf{inr}(a, x)) = \mathsf{inr}(a, h(x))$$

Преобразование $F_A(h) : F_A(X) \to F_A(Y)$ применяет $h : X \to Y$ к продолжению, сохраняя конструктор и данные элемента.

Структурная свёртка списка

```
let rec foldr f z = function
  | [] -> z
  | x :: xs -> f x (foldr f z xs)
```

foldr : $(A \to B \to B) \to B \to \text{List}A \to B$

Свёртка заменяет пустой список значением $z : B$, а конструктор $x :: xs$ — обработчиком f.

Обработчик получает элемент и **уже вычисленный результат** для хвоста.

Вычисление свёртки

$$\mathsf{foldr}\,(+)\,0\,(1 :: 2 :: 3 :: [])$$

$$=_{\text{fold}} 1 + (2 + (3 + 0))$$

$$= 6$$

Вложенные вправо скобки отражают применение обработчика к элементу и результату свёртки хвоста.

$$\mathsf{foldr}\,(-)\,0\,(1 :: 2 :: 3 :: []) = 1 - (2 - (3 - 0)) = 2$$

Разные обработчики одной структуры

```
let sum xs = foldr (+) 0 xs
let length xs = foldr (fun _ n -> 1 + n) 0 xs
let map g xs = foldr (fun x ys -> g x :: ys) [] xs
```

Результатом может быть число или новая структура данных.

$\mathrm{sum} : \mathrm{List}\,\mathbb{Z} \to \mathbb{Z} \qquad \mathrm{length} : \mathrm{List}\,A \to \mathbb{N}$

$\mathrm{map} : (A \to B) \to \mathrm{List}\,A \to \mathrm{List}\,B$

В коде OCaml используются машинные целые, в формулах — математические $\mathbb{Z}$.

Дерево и тип его свёртки

```
type 'a tree =
  | Leaf
  | Node of 'a * 'a tree * 'a tree
```

foldTree : $B \to (A \to B \to B \to B) \to \mathrm{Tree}A \to B$

Свёртке нужны результат типа B для пустого дерева и обработчик метки узла с **двумя рекурсивными результатами**.

Тип обработчика выводится из полей конструктора.

Свёртка дерева

```
let rec fold_tree leaf node = function
  | Leaf -> leaf
  | Node (x, l, r) ->
      node x (fold_tree leaf node l)
             (fold_tree leaf node r)
```

Каждый рекурсивный вызов работает с **собственным поддеревом**.

height = foldTree 0 $(\lambda a\, l\, r.1 + \max(l, r))$

Почему структурная свёртка завершается

Предпосылки

- Вход — конечное индуктивное значение
- Каждый вызов обработчика завершается
- Рекурсивные вызовы получают собственные подструктуры

Доказательство проводится **индукцией по структуре входа**.

Алгебра одного слоя дерева

Из поддеревьев уже получены результаты типа B

$$F_A(B) = \mathbf{1} + A \times B \times B$$

$$\alpha : F_A(B) \to B$$

$$\alpha(\mathsf{inl}()) = leaf$$

$$\alpha(\mathsf{inr}(a, (b_l, b_r))) = node\ a\ b_l\ b_r$$

Алгебра объединяет обработчики пустого дерева и узла, у которого рекурсивные поля заменены **готовыми результатами**.

Для высоты $B = \mathbb{N}$, $leaf = 0$, $node\ a\ l\ r = 1 + \max(l, r)$.

Катаморфизм: общее описание свёртки

$T = \mu X.F(X)$ $\quad$ $\mathsf{in} : F(T) \to T$

$\alpha : F(B) \to B$ $\quad$ $\mathsf{cata}\,\alpha : T \to B$

$$\mathsf{cata}\,\alpha \circ \mathsf{in} = \alpha \circ F(\mathsf{cata}\,\alpha)$$

Рекурсивно сворачиваем дочерние структуры, затем обрабатываем один слой с помощью α.

Для начальной алгебры (T, in) это уравнение определяет катаморфизм единственным образом.

Единственность свёртки списка

Пусть чистая тотальная функция $h : \mathrm{List} A \to B$ удовлетворяет двум уравнениям.

$$h([]) = z \qquad h(x :: xs) = f\ x\ (h(xs))$$

На пустом списке h совпадает с foldr $f\ z$.

Для $x :: xs$ используем совпадение на хвосте и применяем одинаковый обработчик f.

$$h = \mathrm{foldr}\ f\ z$$

Слияние свёртки и последующей обработки

$f : A \to B \to B$ $\quad g : A \to C \to C$ $\quad h : B \to C$

Условия согласованности

$h(z) = w$ $\quad h(f\ x\ b) = g\ x\ (h(b))$

$h \circ \mathsf{foldr}\ f\ z = \mathsf{foldr}\ g\ w$

Для пустого списка используем $h(z) = w$, а для $x :: xs$ — условие согласованности обработчиков и предположение индукции.

Условия проверяем для всех $x : A$ и $b : B$.

Два результата за один обход

Можно одновременно вычислить сумму элементов и длину списка.

$$z = (0, 0)$$

$$f\ x\ (s, n) = (x + s, n + 1)$$

$$\mathsf{foldr}\ f\ z\ xs = (\mathsf{sum}\ xs, \mathsf{length}\ xs)$$

Выбираем **произведение результатов** в качестве носителя алгебры, чтобы вычислять обе компоненты за один обход.

Стоимость такого вычисления складывается из обхода узлов и работы обработчиков с результатами соответствующего размера.

Материал для следующей практики

- Суммы описывают альтернативы, произведения — совместные данные
- Суммы и произведения образуют полукольцо на классах изоморфизма
- Изоморфизм задаётся двумя взаимно обратными программами
- Конструкторы определяют тип обработчиков свёртки
- Типизация, покрытие и завершение — отдельные утверждения

Практика: выбрать представление данных, написать интерпретатор и проверить переходы автомата.

Дополнительный материал

Темы для отдельного обсуждения

Следующие разделы предназначены для самостоятельной работы, отдельных обсуждений и последующих занятий.

Булево и тропическое полукольца

Логические значения

$$(\{\text{false}, \text{true}\}, \vee, \wedge, \text{false}, \text{true})$$

Дизъюнкция объединяет альтернативы, конъюнкция связывает условия.

Длины путей

$$(\mathbb{N} \cup \{\infty\}, \min, +, \infty, 0)$$

Выбираем минимум среди альтернативных путей, а длину составного пути получаем сложением длин его участков.

$$\min(3, 5) + 2 = \min(3 + 2, 5 + 2) = 5$$

Полиномиальные функторы и индуктивность

$$F(X) = \sum_{i=1}^{m} A_i \times X^{k_i} \qquad k_i \in \mathbb{N}$$

Каждый конструктор хранит данные A_i и k_i рекурсивных дочерних значений.

$$F(\mathrm{id}) = \mathrm{id} \qquad F(g \circ h) = F(g) \circ F(h)$$

Для таких конечных полиномиальных функторов в множествах конечные деревья образуют начальную алгебру.

Законы катаморфизмов

$$\mathsf{cata}\,\mathsf{in} = \mathsf{id}$$

$$h \circ \alpha = \beta \circ F(h) \implies h \circ \mathsf{cata}\,\alpha = \mathsf{cata}\,\beta$$

Совместное вычисление двух свёрток

$$\alpha : F(B) \to B \qquad \beta : F(C) \to C$$

$$\delta = \langle \alpha \circ F(\pi_1),\ \beta \circ F(\pi_2) \rangle : F(B \times C) \to B \times C$$

$$\mathsf{cata}\,\delta = \langle \mathsf{cata}\,\alpha, \mathsf{cata}\,\beta \rangle$$

Проекции приводят рекурсивные результаты к нужным носителям B и C.

Списки как формальный ряд

$$\mathsf{List}A \cong \coprod_{n \geq 0} A^n$$

Слагаемое A^n хранит n элементов, а метка n фиксирует длину списка.

$$L(z) = 1 + z + z^2 + \cdots = \frac{1}{1 - z}$$

В этом тождестве формальных степенных рядов переменная z отмечает число позиций для элементов.

Списки над одноэлементным типом соответствуют натуральным числам — своим длинам.

Производная типа: одно отверстие

$$D(c) = \mathbf{0} \qquad D(A) = \mathbf{1}$$

$$D(F + G) = D(F) + D(G)$$

$$D(F \times G) = D(F) \times G + F \times D(G)$$

$$D(A \times A) \cong A + A$$

Контекст пары состоит из оставшегося элемента и метки, указывающей положение отверстия: слева или справа.

Контекст списка и zipper

$$D(\mathsf{List}A) \cong \mathsf{List}A \times \mathsf{List}A$$

Отверстие делит список на префикс и суффикс.

$$\mathsf{Zipper}\,A = \mathsf{List}A \times A \times \mathsf{List}A$$

К префиксу и суффиксу добавляем значение в фокусе, храня левую часть **в обратном порядке**.

Сдвиг к соседу и замена фокуса занимают $O(1)$, а поиск далёкой позиции и восстановление списка — линейное время в худшем случае.

Класс типов как передаваемый словарь

```
type 'a printer = 'a -> string
let print_pair show_a show_b (a, b) =
  "(" ^ show_a a ^ ", " ^ show_b b ^ ")"
```

Обобщённая функция получает операции для конкретных типов через аргументы.

$$\text{printPair} : (A \to \text{String}) \to (B \to \text{String}) \to A \times B \to \text{String}$$

Классы типов автоматизируют выбор и передачу словарей с операциями, законы которых доказываются для конкретной реализации.

Комбинирование результатов и моноид

$$\diamond : B \times B \to B \qquad e : B$$

$$(x \diamond y) \diamond z = x \diamond (y \diamond z)$$

$$e \diamond x = x = x \diamond e$$

Ассоциативная операция с нейтральным элементом позволяет объединять результаты обработки данных.

Примеры: сложение целых с нулём и конкатенация строк с пустой строкой.

Перестановка строк при конкатенации меняет результат.

Foldable и форма данных

`Foldable` сворачивает последовательность элементов согласно выбранному обходу.

Поскольку разные деревья могут давать **одинаковую последовательность меток**, сведения об их форме берём из конструкторов.

$$\text{inorder}(\text{Node } 1 \text{ Leaf } (\text{Node } 2 \text{ Leaf Leaf})) = (1, 2)$$

$$\text{inorder}(\text{Node } 2 \ (\text{Node } 1 \text{ Leaf Leaf}) \text{ Leaf}) = (1, 2)$$

Обработчики всех конструкторов в структурном катаморфизме позволяют восстанавливать и сериализовать дерево.

Полиморфизм и свойства программ

$$r : \forall A.\mathrm{List}A \to \mathrm{List}A$$

В чистой тотальной параметрической модели r выбирает, повторяет и переставляет элементы, опираясь на **структуру списка**.

$$\mathrm{map}\, g \circ r_A = r_B \circ \mathrm{map}\, g \qquad (g : A \to B)$$

Такую связь поведения с типом называют теоремой из типа, предпосылки которой разберём при изучении System F.

Представление ADT во время исполнения

Логическая сумма описывает выбор конструктора, а физическое представление его данных определяется языком, компилятором и типами полей.

Представление может использовать явные метки, указатели или свободные комбинации битов.

Пример документированной оптимизации

В Rust при `T: Sized` тип `Option<&T>` имеет те же размер и выравнивание, что и `&T`.

Стоимость размещения и обработки проверяется для конкретного представления.